



Laboratório 2: criptografia

16/09/2025

1 Ferramenta OpenSSL

O OpenSSL¹ é uma biblioteca de criptografia de código aberto que implementa o protocolo TLS, além de fornecer ferramentas para gerar chaves, certificados, cifrar, decifrar e assinar e verificar mensagens. No Ubuntu, você pode instalar o OpenSSL com o comando: `sudo apt install openssl`. Na Listagem 1 são ilustrados os comandos para criação de uma chave simétrica, para cifrar e decifrar um arquivo com uma chave simétrica usando o OpenSSL.

Listagem 1: Criptografia simétrica de arquivos

```
# Usando o PBKDF2 para criar uma chave derivada de uma senha mestra e cifrar a mensagem
openssl enc -aes-256-cbc -salt -in mensagem.txt -out mensagem.enc -iter 1000 -pbkdf2 -k senha

# Decifrar o arquivo com a senha mestra
openssl enc -d -aes-256-cbc -in mensagem.enc -out mensagem.dec -iter 1000 -pbkdf2 -k senha

# Gerando a chave simétrica de 32 bytes (256 bits) e salvando em chave.txt no formato base64
openssl rand -base64 32 > chave.txt

# Cifrar o arquivo mensagem.txt com a chave simétrica
openssl enc -aes-256-cbc -in mensagem.txt -out mensagem.enc -iter 1000 -pass file:chave.txt

# Decifrar o arquivo cifrado com a chave simétrica
openssl enc -d -aes-256-cbc -in mensagem.enc -out mensagem.dec -iter 1000 -pass file:chave.txt
```

O OpenSSL também pode ser usado para gerar chaves assimétricas, assinar e verificar mensagens. Na Listagem 2 são ilustrados os comandos para gerar um par de chaves RSA, assinar e verificar uma mensagem.

Listagem 2: Criptografia assimétrica para assinatura digital

```
# Gerar um par de chaves RSA
openssl genrsa -out chave-privada.pem 2048
openssl rsa -in chave-privada.pem -pubout -out chave-publica.pem

# Gerando um par de chaves ECC com OpenSSL
openssl ecparam -name prime256v1 -genkey -noout -out chave-privada-ec.pem
openssl ec -in chave-privada-ec.pem -pubout -out chave-publica-ec.pem

# Imprimindo a chave no formato PEM
openssl rsa -in chave-privada.pem -text -noout
openssl ec -in chave-privada-ec.pem -text -noout

# Assinar a mensagem com a chave privada
openssl dgst -sha256 -sign chave-privada.pem -out mensagem.sig mensagem.txt

# Verificar a assinatura com a chave pública
openssl dgst -sha256 -verify chave-publica.pem -signature mensagem.sig mensagem.txt
```

¹<https://www.openssl.org>

Para cifrar um arquivo com uma chave pública e decifrar com a chave privada, você pode usar o OpenSSL como ilustrado na Listagem 3. **Porém, com a criptografia assimétrica, o tamanho do arquivo a ser cifrado não pode ser maior que o tamanho da chave RSA.**

Listagem 3: Criptografia assimétrica para cifrar arquivos não maiores que a chave RSA

```
# Cifrar o arquivo com a chave pública
openssl pkeyutl -encrypt -pubin -inkey chave-publica.pem -in mensagem.txt -out mensagem.enc

# Decifrar o arquivo com a chave privada
openssl pkeyutl -decrypt -inkey chave-privada.pem -in mensagem.enc -out mensagem.dec
```

Caso você queira cifrar um arquivo maior que o tamanho da chave RSA, você pode usar o OpenSSL para cifrar o arquivo com uma chave simétrica e cifrar a chave simétrica com a chave pública. Na Listagem 4 são ilustrados os comandos para cifrar e decifrar um arquivo com uma chave simétrica e cifrar e decifrar a chave simétrica com uma chave pública.

Listagem 4: Criptografia assimétrica para cifrar arquivos grandes

```
# Criando um arquivo com dados aleatórios e tamanho de 1MB
dd if=/dev/urandom of=grande.dat bs=1M count=1

# Gerar a chave simétrica
openssl rand -base64 32 > chave.txt

# Cifrar o arquivo com a chave simétrica
openssl enc -aes-256-cbc -in grande.dat -out grande.enc -iter 1000 -pass file:chave.txt

# Cifrar a chave simétrica com a chave pública
openssl pkeyutl -encrypt -pubin -inkey chave-publica.pem -in chave.txt -out chave.enc

# Decifrar a chave simétrica com a chave privada
openssl pkeyutl -decrypt -inkey chave-privada.pem -in chave.enc -out chave.txt

# Decifrar o arquivo com a chave simétrica
openssl enc -d -aes-256-cbc -in grande.enc -out saida.dat -iter 1000 -pass file:chave.txt
```

1.1 Exercício em dupla

1. Um componente da dupla deve baixar o PDF com os slides da aula de criptografia, gerar uma chave simétrica de 256 bits e cifrar o PDF com essa chave. O outro componente da dupla deve decifrar o PDF com a chave simétrica para obter o conteúdo original.
2. Um componente da dupla deve baixar o PDF com os slides da aula de criptografia e deve usar a criptografia de chave pública para cifrar o PDF. O outro componente da dupla deve decifrar o PDF e verificar se o conteúdo está íntegro, ou seja, se o conteúdo decifrado é igual ao conteúdo original e se a assinatura digital é válida (usando a chave pública do primeiro componente).

Nota

É importante que a dupla documente todos os passos executados, por cada componente da dupla e na ordem exata, para concluir a tarefa. Isso pode ser feito em um arquivo texto simples, como o `Readme.md`, dentro do diretório do exercício.

2 GNU Privacy Guard (GPG)

O *GNU Privacy Guard* (GPG)² é uma ferramenta de criptografia que permite a criação de chaves públicas e privadas para a troca segura de mensagens. Neste laboratório, você irá aprender a criar um par de chaves, exportar a chave pública, importar a chave pública de um colega, criptografar e descriptografar mensagens.

Gerar um par de chaves

Para gerar um par de chaves, execute o comando abaixo:

```
# Gera um par de chaves
gpg --full-generate-key

# Escolha a opção 9 (ECC) sign and encrypt e a curva 1 (Curve 25519)
```

Listar chaves

As chaves são armazenadas em um banco de dados dentro do diretório ~/.gnupg. Para listar as chaves, execute o comando abaixo:

```
# Lista todas as chaves armazenadas no banco de dados do GPG
gpg --list-keys

# Um exemplo de saída para o comando acima:
/home/juca/.gnupg/pubring.kbx
-----
pub   ed25519 2024-10-18 [SC]
      0C8A1F74BAB9AE2C00E616D92D5C437ECD19BE00
uid           [ultimate] Juca Simas <juca@example.org>
sub   cv25519 2024-10-18 [E]

# Para listar somente as chaves privadas
gpg --list-secret-keys
```

Troca de chaves

Para que alguém possa enviar uma mensagem cifrada para você, ou que possa verificar a autenticidade de uma mensagem assinada por você, é necessário que você envie a sua chave pública. Para exportar a chave pública, execute o comando abaixo:

```
# Exporta a chave pública com o ID juca@example.org
gpg --armor --output juca-public.key --export juca@example.org
```

Agora, envie o arquivo juca-public.key para um colega. Para importar a chave pública de um colega (Bob), execute o comando abaixo:

```
# Importa a chave pública do colega Bob
gpg --import bob-public.key
```

²<https://www.gnupg.org/>

Cifrar e decifrar arquivos

Para cifrar um arquivo é necessário indicar o destinatário da mensagem (parâmetro `-r`), podendo ser o e-mail ou o ID da chave pública que você importou do seu colega. O arquivo cifrado pode ser representado em formato ASCII (parâmetro `-a`), ou em formato binário. Para cifrar um arquivo, execute o comando abaixo:

```
# Para cifrar o arquivo chamado slides.pdf, irá gerar um arquivo chamado slides.pdf.gpg
gpg -r bob@example.org -e slides.pdf
```

Bob, ao receber o arquivo cifrado, poderá decifrá-lo com o comando abaixo:

```
# Para decifrar o arquivo slides.pdf.gpg, irá gerar um arquivo chamado slides.pdf
gpg --output slides.pdf -d slides.pdf.gpg
```

Assinar e verificar arquivos

A assinatura é armazenada um arquivo com a extensão `.asc`, caso use o parâmetro `-a`. Caso contrário, será gerado um arquivo com a extensão `.gpg`. O arquivo gerado é uma combinação do arquivo original com a assinatura. Para assinar um arquivo, execute o comando abaixo:

```
# Saída: slides.pdf.gpg que consiste no arquivo slides.pdf com a assinatura embutida
gpg -s slides.pdf

# É possível gerar assinaturas que não estão embutidas no arquivo original
gpg -o slides.pdf.sig -a -b slides.pdf
```

Para verificar a autenticidade de um arquivo assinado, execute o comando abaixo:

```
# Verifica a autenticidade do arquivo slides.pdf.gpg. A saída será o arquivo slides.pdf e a
  mensagem Good signature, caso a assinatura seja válida
gpg -v slides.pdf.gpg

# Verifica a autenticidade do arquivo slides.pdf e a assinatura slides.pdf.sig
gpg --verify slides.pdf.sig slides.pdf
```

Cifrar e assinar arquivos

É possível cifrar e assinar um arquivo ao mesmo tempo. Para isso, execute o comando abaixo:

```
# Cifrar o arquivo com a chave pública de Bob e assinar com a chave privada de Juca
gpg -es -r bob@example.org -u juca@example.org slides.pdf

# Decifrar o arquivo e verificar a assinatura
gpg -o slides.pdf -d slides.pdf.gpg
```

3 Acesso remoto usando SSH e chaves

O SSH (Secure Shell) é um protocolo de rede que permite o acesso remoto seguro a sistemas. Ele utiliza criptografia para proteger a comunicação entre o cliente e o servidor, garantindo confidencialidade, integridade e autenticidade dos dados transmitidos.

Faremos uso do Docker para subir um container com o servidor SSH. Para isso, criaremos uma imagem Docker personalizada que conterá o servidor SSH.

3.1 Criando a imagem Docker com o servidor SSH

Crie um arquivo chamado Dockerfile com o seguinte conteúdo:

Listagem 5: Dockerfile para criar imagem com servidor SSH

```
FROM ubuntu:latest
RUN apt-get update && apt-get install -y openssh-server && \
    mkdir /var/run/sshd && \
    echo 'root:senha' | chpasswd && \
    sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && \
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication yes/' /etc/ssh/sshd_config
EXPOSE 22
ENTRYPOINT ["/usr/sbin/sshd", "-D"]
```

Para construir a imagem Docker, execute o seguinte comando no terminal:

Listagem 6: Comando para construir a imagem Docker

```
docker build -t servidor-ssh .
```

Após a construção da imagem, você pode executar um container com o servidor SSH usando o seguinte comando:

Listagem 7: Comando para executar o container com o servidor SSH

```
docker run -d -p 2222:22 --rm --name ssh-container servidor-ssh
```

3.2 Acessando o servidor SSH com senha

Para acessar o servidor SSH em execução no container, você pode usar o seguinte comando:

Listagem 8: Comando para acessar o servidor SSH

```
ssh root@localhost -p 2222
```

Você será solicitado a inserir a senha do usuário root, que é senha.

3.3 Gerando chaves SSH

Para acessar o servidor SSH de forma mais segura, é recomendado o uso de chaves SSH. Para isso, você deve gerar um par de chaves SSH no seu computador local. Execute o seguinte comando:

Listagem 9: Comando para gerar chaves SSH

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519
```

Isso irá gerar um par de chaves SSH: uma chave privada (id_ed25519) e uma chave pública (id_ed25519.pub).

3.4 Copiando a chave pública para o servidor SSH

Para permitir o acesso ao servidor SSH usando a chave pública, você deve copiar a chave pública para o servidor. Execute o seguinte comando:

Listagem 10: Comando para copiar a chave pública para o servidor SSH

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub -p 2222 root@localhost
```

Isso irá adicionar a chave pública ao arquivo `/root/.ssh/authorized_keys` no servidor SSH, permitindo o acesso sem a necessidade de senha. Você também pode copiar manualmente o conteúdo de `id_ed25519.pub` para `/root/.ssh/authorized_keys` no servidor.

3.5 Acessando o servidor SSH com chave privada

Agora você pode acessar o servidor SSH usando a chave privada, sem precisar digitar a senha. Execute o seguinte comando:

Listagem 11: Comando para acessar o servidor SSH com chave privada

```
ssh -i ~/.ssh/id_ed25519 root@localhost -p 2222
```

Você deve conseguir acessar o servidor SSH sem precisar digitar a senha.

3.6 Criando uma imagem Docker com persistência de dados e sem acesso root com senha

O problema da imagem Docker anterior é que ela não persiste os dados, ou seja, se você parar o container e iniciar um novo, as chaves SSH não estarão mais disponíveis. No Dockerfile abaixo iremos criar uma imagem mais segura que anterior, por não permitirá o acesso ao usuário root com senha, e também persistirá os dados do usuário juca e a chave pública `id_ed25519.pub`. Crie um arquivo chamado `Dockerfile` com o seguinte conteúdo:

Listagem 12: Dockerfile para criar imagem com servidor SSH e persistência de dados

```
FROM ubuntu:latest
RUN apt-get update && apt-get install -y openssh-server && \
    mkdir /var/run/ssh && \
    useradd -m juca && \
    echo 'juca:senha' | chpasswd && \
    mkdir -p /home/juca/.ssh && \
    chmod 700 /home/juca/.ssh && \
    chown juca:juca /home/juca/.ssh && \
    sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin no/' /etc/ssh/sshd_config && \
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config

# Copiando a chave pública para o diretório do usuário juca
# Esse arquivo deve ser criado previamente com o comando ssh-keygen e estar no mesmo diretório do Dockerfile
COPY id_ed25519.pub /home/juca/.ssh/authorized_keys

RUN chmod 600 /home/juca/.ssh/authorized_keys && \
    chown juca:juca /home/juca/.ssh/authorized_keys

EXPOSE 22
ENTRYPOINT ["/usr/sbin/sshd", "-D"]
```

Para construir a imagem Docker, execute o seguinte comando no terminal:

Listagem 13: Comando para construir a imagem Docker com persistência de dados

```
docker build -t servidor-ssh .
```

Após a construção da imagem, você pode executar um container com o servidor SSH usando o seguinte comando:

Listagem 14: Comando para executar o container com o servidor SSH e persistência de dados

```
docker run -d -p 2222:22 --rm --name ssh-container
```

Agora, você pode acessar o servidor SSH com o usuário `juca` com a chave privada que você gerou anteriormente. Execute o seguinte comando:

Listagem 15: Comando para acessar o servidor SSH com persistência de dados

```
ssh -i ~/.ssh/id_ed25519 juca@localhost -p 2222
```

3.7 Exercício: Assinando *commits* com git

Siga as instruções no tutorial “*Gerenciar a verificação de assinatura de commit*”³ do GitHub para configurar a verificação de assinatura de *commit* por meio de chaves SSH. Use o par de chaves que você criou no exercício anterior. Por fim, faça um *commit* em um repositório do GitHub para demonstrar que você conseguiu configurar a verificação de assinatura de *commit*.

Em <https://github.com/emersonmello/modelos-latex/commits/master> você pode ver um exemplo de *commit* assinado com a chave SSH e verificado pelo GitHub.

❗ Cuidado

Depois de fazer este exercício, você deve remover a chave SSH da sua conta no GitHub, uma vez que essa chave foi criada nos computadores compartilhados no laboratório do campus. Contudo, em seu computador pessoal é uma boa ideia usar chaves SSH para assinar seus *commits* ou mesmo para interagir com o GitHub, como por exemplo, para fazer *push* e *pull* de repositórios.

4 Implementação da cifra de César em Java

A cifra de César é uma cifra de substituição simples, onde cada letra do texto é deslocada um número fixo de posições no alfabeto (para direita ou esquerda). Na Equação 1 e Equação 2 são apresentadas as fórmulas para criptografar e descriptografar uma letra com a cifra de César e com deslocamento à esquerda.

$$\mathcal{E}_n(x) = (x - n) \mod 26 \quad (1)$$

$$\mathcal{D}_n(x) = (x + n) \mod 26 \quad (2)$$

, sendo x a posição da letra no alfabeto e n o deslocamento.

4.1 Exercício em dupla ou individual

Crie um projeto Java com gradle para cada um dos exercícios abaixo. O projeto deve conter um arquivo `Readme.md` com as instruções para compilar e executar o programa.

1. Implementar a cifra de César em Java. O programa deve receber, como argumentos de linha de comando, o deslocamento e o nome do arquivo de texto a ser criptografado. O programa deve ler o arquivo de texto, cifrar o texto e imprimir o texto cifrado na tela.

- Atente-se para o caminho do arquivo, que pode ser diferente quando executado no terminal (`./gradlew run`) ou no IDE.

³<https://docs.github.com/pt/authentication/managing-commit-signature-verification>

2. Implementar um programa que fará uso da força bruta⁴ para decifrar o arquivo. O programa deve receber, como argumentos de linha de comando, o nome do arquivo de texto cifrado e a técnica de força bruta. As técnicas de força bruta que o programa deve implementar são:

- (a) **Busca manual com ajuda do usuário.** Em cada tentativa de decifrar o texto, o programa deve imprimir o texto decifrado e perguntar ao usuário se o texto está correto. O usuário deve responder com S ou N. Se o usuário responder S, o programa deve imprimir o deslocamento usado na criptografia e encerrar. Se o usuário responder N, o programa deve tentar o próximo deslocamento;
- (b) **Busca automática com uso de dicionário de palavras.** Em <https://www.ime.usp.br/~pf/dicios/> é disponibilizado o arquivo `br-utf8.txt` com uma lista com mais de 260.000 palavras em português. O programa deve tentar todos os deslocamentos possíveis e, para cada deslocamento, decifrar o texto cifrado. O programa deve contar quantas palavras do texto decifrado estão presentes no arquivo `br-utf8.txt`. Se pelo menos 50% das palavras do texto decifrado estiverem contidas no arquivo de dicionário, então o programa deve imprimir o texto decifrado e perguntar ao usuário se o texto está correto. Se o usuário responder S, o programa deve imprimir o deslocamento usado na criptografia e encerrar. Se o usuário responder N, o programa deve tentar o próximo deslocamento.

Nota

Ler o arquivo de palavras e armazenar em um `HashSet` pode ser uma estratégia eficiente para verificar se uma palavra está no arquivo, pois a busca em um `HashSet` é $O(1)$ (algo que irão aprender em Estrutura de Dados).

 Documento licenciado sob [Creative Commons “Atribuição 4.0 Internacional”](https://creativecommons.org/licenses/by/4.0/).

⁴Existem outras técnicas mais eficientes, como a análise de frequência de letras https://pt.wikipedia.org/wiki/Frequ%C3%AAncia_de_letras, <http://www.numaboa.com.br/criptografia/criptoanalise/309-Ferramenta-de-frequencia>, <http://www.numaboa.com.br/criptografia/criptoanalise/1051-exemplo>